

Remove nodiscard annotations from the standard library specification

Abstract

This is a proposal for removing `[[nodiscard]]` annotations from the standard library specification. These annotations

1. are pure QoI
2. specify nothing, so it's questionable to put them into a *specification*
3. can be made available uniformly without spending any WG21 time
4. need implementation analysis and implementation experience to be made available uniformly
5. are sometimes highly non-obvious and subject to change based on deployment experience

All that makes them a bad fit for spending all of LEWG time, LWG time, and plenary time. The vast majority of time and effort for the exercise is spent on (4), with occasional (and possibly sometimes surprising and urgent) (5). Those parts are not something a standards committee can meaningfully participate in, and the remaining parts are bad and rigid overhead, and a waste of the time of the committee.

Various arguments for putting the annotations into the specification

"But I want the annotations to be 'portable' so that my code doesn't break when compiled with an implementation that has the annotations, when the one I used before doesn't have them"

While that's a nice idea,

1. your code is portably broken, regardless of whether all implementations diagnose it
2. you can get all the implementations to warn you by writing a recommendation for when and where they should, and by sending patches to them
3. the recommendation and the patches can be produced without spending LEWG/LWG/WG21 time, and can be done better and faster without spending that time

"But the places where we currently have a `[[nodiscard]]` are bad bugs unless diagnosed"

Yes, they are. But that's QoI like all other things where failing to do something is (usually) a bad bug. Unless we know something to be really always bad, we risk wasting time chasing a good specification and running into false positives. Is a discarded call to `std::async` always a bug? Depends on whether you ever actually evaluate the call.

"These annotations are helpful both to users and implementation vendors"

Of course they are. That doesn't mean going through all of our procedural pipes is an appropriate way to accomplish that.

For every proposal proposing to add more of these annotations, we *need* to require such a proposal to

1. look at whether STLSTL (you know, MSVC stdlib) already annotates the entity in case, and if not, find out why not
2. run it by the other implementation vendors to have them check whether they think the annotation is feasible and viable

If we don't, we are shooting in the dark, and those things will eventually happen anyway, when implementation vendors consider adding the newly proposed annotation, and then we may need to backtrack and redo things.

Why waste all that time going through the committee? Why not edit a recommendation document, have vendors look at it and give their feedback on it, look at STLSTL, and send patches to it and the others? You'll get the same implementation vendor feedback, same "portability", and you'll get it faster, without spending the time of WG21, or its subgroups.

What to do instead

As suggested on the reflector,

1. rip out the `[[nodiscard]]` annotations from the standard

2. don't add more
3. create a separate document containing recommendations where to put `[[nodiscard]]` in a standard library implementation
4. have implementation vendors look at the revisions of that document periodically
5. edit the document when you have done analysis and implementation work, not otherwise
6. and sure, form either an ad-hoc ARG or a real ARG with a mailing list for people interested in doing this work

Side note about sending patches to implementations

All three of our major library implementations are open source (or even Free Software, for those who insist on terminology). And *none* of them require going through red tape hoops to accept patches, not even libstdc++, any more.

If you bothered implementing a `[[nodiscard]]` addition, did you test it? If you did, surely you tested it with an existing testsuite of an implementation? If you didn't, why are we listening to your suggestion to add it? The implementation and testing effort is straightforward, and mandatory for the desired end effect to materialize. So the world cannot skip it, even if ivory tower proposal authors might think it's not their problem.

Wording

Drafting note: the examples are okay, no need to remove the `[[nodiscard]]` from those.

In `[basic.stc.dynamic.general]/2`, edit as follows:

```
[[nodiscard]] void* operator new(std::size_t);
[[nodiscard]] void* operator new(std::size_t, std::align_val_t);
void operator delete(void*) noexcept;
void operator delete(void*, std::size_t) noexcept;
void operator delete(void*, std::align_val_t) noexcept;
void operator delete(void*, std::size_t, std::align_val_t) noexcept;
[[nodiscard]] void* operator new[](std::size_t);
[[nodiscard]] void* operator new[](std::size_t, std::align_val_t);
```

In `[new.syn]`, edit as follows:

```
.
.
// 17.6.5, pointer optimization barrier
template [[nodiscard]] constexpr T* launder(T* p) noexcept;
.
.
// 17.6.3, storage allocation and deallocation
[[nodiscard]] void* operator new(std::size_t size);
[[nodiscard]] void* operator new(std::size_t size, std::align_val_t alignment);
[[nodiscard]] void* operator new(std::size_t size, const std::nothrow_t&) noexcept;
[[nodiscard]] void* operator new(std::size_t size, std::align_val_t alignment, const std::nothrow_t&) noexcept;
.
.
[[nodiscard]] void* operator new[](std::size_t size);
[[nodiscard]] void* operator new[](std::size_t size, std::align_val_t alignment);
[[nodiscard]] void* operator new[](std::size_t size, const std::nothrow_t&) noexcept;
[[nodiscard]] void* operator new[](std::size_t size, std::align_val_t alignment, const std::nothrow_t&) noexcept;
.
.
[[nodiscard]] void* operator new (std::size_t size, void* ptr) noexcept;
[[nodiscard]] void* operator new[](std::size_t size, void* ptr) noexcept;
```

In `[new.delete.single]`, edit as follows:

```
[[nodiscard]] void* operator new(std::size_t size);
[[nodiscard]] void* operator new(std::size_t size, std::align_val_t alignment);
.
.
[[nodiscard]] void* operator new(std::size_t size, const std::nothrow_t&) noexcept;
[[nodiscard]] void* operator new(std::size_t size, std::align_val_t alignment,
const std::nothrow_t&) noexcept;
```

In `[new.delete.array]`, edit as follows:

```
[[nodiscard]] void* operator new[](std::size_t size);
[[nodiscard]] void* operator new[](std::size_t size, std::align_val_t alignment);
.
.
[[nodiscard]] void* operator new[](std::size_t size, const std::nothrow_t&) noexcept;
[[nodiscard]] void* operator new[](std::size_t size, std::align_val_t alignment,
const std::nothrow_t&) noexcept;
```

In [new.delete.placement], edit as follows:

```
[[nodiscard]] void* operator new(std::size_t size, void* ptr) noexcept;
.
.
[[nodiscard]] void* operator new[](std::size_t size, void* ptr) noexcept;
```

In [ptr.laundry], edit as follows:

```
template<class T> [[nodiscard]] constexpr T* laundry(T* p) noexcept;
```

In [memory.syn], edit as follows:

```
template<size_t N, class T>
[[nodiscard]] constexpr T* assume_aligned(T* ptr);
```

In [ptr.align], edit as follows:

```
template<size_t N, class T>
[[nodiscard]] constexpr T* assume_aligned(T* ptr);
```

In [allocator.traits.general], edit as follows:

```
.
.
[[nodiscard]] static constexpr pointer allocate(Alloc& a, size_type n);
[[nodiscard]] static constexpr pointer allocate(Alloc& a, size_type n,
const_void_pointer hint);
```

In [allocator.traits.members], edit as follows:

```
[[nodiscard]] static constexpr pointer allocate(Alloc& a, size_type n);
.
.
[[nodiscard]] static constexpr pointer allocate(Alloc& a, size_type n, const_void_pointer hint);
```

In [allocator.traits.other], edit as follows:

```
template<class Allocator>
[[nodiscard]] constexpr allocation_result<typename allocator_traits<Allocator>::pointer>
allocate_at_least(Allocator& a, size_t n);
```

In [default.allocator.general], edit as follows:

```
[[nodiscard]] constexpr T* allocate(size_t n);
[[nodiscard]] constexpr allocation_result allocate_at_least(size_t n);
```

In [allocator.members], edit as follows:

```
[[nodiscard]] constexpr T* allocate(size_t n);
.
.
[[nodiscard]] constexpr allocation_result allocate_at_least(size_t n);
```

In [mem.res.class.general], edit as follows:

```
[[nodiscard]] void* allocate(size_t bytes, size_t alignment = max_align);
```

In [mem.res.public], edit as follows:

```
[[nodiscard]] void* allocate(size_t bytes, size_t alignment = max_align);
```

In [mem.poly.allocator.class.general], edit as follows:

```
[[nodiscard]] Tp* allocate(size_t n);
.
.
[[nodiscard]] void* allocate_bytes(size_t nbytes, size_t alignment = alignof(max_align_t));
.
.
template<class T> [[nodiscard]] T* allocate_object(size_t n = 1);
.
.
template<class T, class... CtorArgs> [[nodiscard]] T* new_object(CtorArgs&&... ctor_args);
```

In [mem.poly.allocator.mem], edit as follows:

```

+{nodiscard}+ Tp* allocate(size_t n);
.
+{nodiscard}+ void* allocate_bytes(size_t nbytes, size_t alignment = alignof(max_align_t));
.
.
template<class T>
+{nodiscard}+ T* allocate_object(size_t n = 1);
.
.
template<class T, class... CtorArgs>
+{nodiscard}+ T* new_object(CtorArgs&&... ctor_args);

```

In [allocator.adaptor.syn], edit as follows:

```

+{nodiscard}+ pointer allocate(size_type n);
+{nodiscard}+ pointer allocate(size_type n, const_void_pointer hint);

```

In [allocator.adaptor.members], edit as follows:

```

+{nodiscard}+ pointer allocate(size_type n);
.
+{nodiscard}+ pointer allocate(size_type n, const_void_pointer hint);

```

In [stacktrace.basic.overview], edit as follows:

```

+{nodiscard}+ bool empty() const noexcept;

```

In [stacktrace.basic.obs], edit as follows:

```

+{nodiscard}+ bool empty() const noexcept;

```

In [forward], edit as follows:

```

template<class T, class U>
+{nodiscard}+ constexpr auto forward_like(U&& x) noexcept -> see below ;

```

In [basic.string.general], edit as follows:

```

+{nodiscard}+ constexpr bool empty() const noexcept;

```

In [string.capacity], edit as follows:

```

+{nodiscard}+ constexpr bool empty() const noexcept;

```

In [string.view.template.general], edit as follows:

```

+{nodiscard}+ constexpr bool empty() const noexcept;

```

In [string.view.capacity], edit as follows:

```

+{nodiscard}+ constexpr bool empty() const noexcept;

```

In [container.node.overview], edit as follows:

```

+{nodiscard}+ bool empty() const noexcept;

```

In [container.node.observers], edit as follows:

```

+{nodiscard}+ bool empty() const noexcept;

```

In [array.overview], edit as follows:

```

+{nodiscard}+ constexpr bool empty() const noexcept;

```

In [deque.overview], edit as follows:

```

+{nodiscard}+ bool empty() const noexcept;

```

In [forward.list.overview], edit as follows:

```

+{nodiscard}+ bool empty() const noexcept;

```

In [list.overview], edit as follows:

```

+{nodiscard}+ bool empty() const noexcept;

```

In [vector.overview], edit as follows:

```
[[nodiscard]] constexpr bool empty() const noexcept;
```

In [vector.bool], edit as follows:

```
[[nodiscard]] constexpr bool empty() const noexcept;
```

In [map.overview], edit as follows:

```
[[nodiscard]] bool empty() const noexcept;
```

In [multimap.overview], edit as follows:

```
[[nodiscard]] bool empty() const noexcept;
```

In [set.overview], edit as follows:

```
[[nodiscard]] bool empty() const noexcept;
```

```
[[nodiscard]] bool empty() const noexcept;
```

In [unord.map.overview], edit as follows:

```
[[nodiscard]] bool empty() const noexcept;
```

In [unord.multimap.overview], edit as follows:

```
[[nodiscard]] bool empty() const noexcept;
```

In [unord.set.overview], edit as follows:

```
[[nodiscard]] bool empty() const noexcept;
```

In [unord.multiset.overview], edit as follows:

```
[[nodiscard]] bool empty() const noexcept;
```

In [queue.defn], edit as follows:

```
[[nodiscard]] bool empty() const { return c.empty(); }
```

In [priorityqueue.overview], edit as follows:

```
[[nodiscard]] bool empty() const { return c.empty(); }
```

In [stack.defn], edit as follows:

```
[[nodiscard]] bool empty() const { return c.empty(); }
```

In [span.overview], edit as follows:

```
[[nodiscard]] constexpr bool empty() const noexcept;
```

In [span.obs], edit as follows:

```
[[nodiscard]] constexpr bool empty() const noexcept;
```

In [iterator.synopsis], edit as follows:

```
template<class C> [[nodiscard]] constexpr auto empty(const C& c) -> decltype(c.empty());  
template<class T, size_t N> [[nodiscard]] constexpr bool empty(const T (&array)[N]) noexcept;  
template<class E> [[nodiscard]] constexpr bool empty(initializer_list<E> il) noexcept;
```

In [iterator.range], edit as follows:

```
template<class C> [[nodiscard]] constexpr auto empty(const C& c) -> decltype(c.empty());  
.  
.  
template<class T, size_t N> [[nodiscard]] constexpr bool empty(const T (&array)[N]) noexcept;  
.  
.  
template<class E> [[nodiscard]] constexpr bool empty(initializer_list<E> il) noexcept;
```

In [range.subrange.general], edit as follows:

```
constexpr I begin() const requires copyable<I>;
[[nodiscard]] constexpr I begin() requires (!copyable<I>);
.
.
[[nodiscard]] constexpr subrange next(iter_difference_t<I> n = 1) const &
requires forward_iterator<I>;
[[nodiscard]] constexpr subrange next(iter_difference_t<I> n = 1) &&;
[[nodiscard]] constexpr subrange prev(iter_difference_t<I> n = 1) const
requires bidirectional_iterator<I>;
```

In [range.subrange.access], edit as follows:

```
[[nodiscard]] constexpr I begin() requires (!copyable<I>);
.
.
[[nodiscard]] constexpr subrange next(iter_difference_t<I> n = 1) const &
requires forward_iterator<I>;
.
.
[[nodiscard]] constexpr subrange next(iter_difference_t<I> n = 1) &&;
.
[[nodiscard]] constexpr subrange prev(iter_difference_t<I> n = 1) const
requires bidirectional_iterator<I>;
```

In [bit.syn], edit as follows:

```
template<class T>
[[nodiscard]] constexpr T rotl(T x, int s) noexcept;
template<class T>
[[nodiscard]] constexpr T rotr(T x, int s) noexcept;
```

In [bit.rotate], edit as follows:

```
template<class T>
[[nodiscard]] constexpr T rotl(T x, int s) noexcept;
.
.
template<class T>
[[nodiscard]] constexpr T rotr(T x, int s) noexcept;
```

In [fs.class.path.general], edit as follows:

```
[[nodiscard]] bool empty() const noexcept;
```

In [fs.path.query], edit as follows:

```
[[nodiscard]] bool empty() const noexcept;
```

In [re.results.general], edit as follows:

```
[[nodiscard]] bool empty() const;
```

In [re.results.size], edit as follows:

```
[[nodiscard]] bool empty() const;
```

In [stoptoken.general], edit as follows:

```
[[nodiscard]] bool stop_requested() const noexcept;
[[nodiscard]] bool stop_possible() const noexcept;
[[nodiscard]] friend bool operator==(const stop_token& lhs, const stop_token& rhs) noexcept;
```

In [stoptoken.mem], edit as follows:

```
[[nodiscard]] bool stop_requested() const noexcept;
.
.
[[nodiscard]] bool stop_possible() const noexcept;
```

In [stoptoken.nonmembers], edit as follows:

```
[[nodiscard]] friend bool operator==(const stop_token& lhs, const stop_token& rhs) noexcept;
```

In [stopsource.general], edit as follows:

```
[[nodiscard]] stop_token get_token() const noexcept;
[[nodiscard]] bool stop_possible() const noexcept;
```

```

[[nodiscard]] bool stop_requested() const noexcept;
.
.
[[nodiscard]] friend bool
operator==(const stop_source& lhs, const stop_source& rhs) noexcept;

```

In [stopsource.mem], edit as follows:

```

[[nodiscard]] stop_token get_token() const noexcept;
.
.
[[nodiscard]] bool stop_possible() const noexcept;
.
.
[[nodiscard]] bool stop_requested() const noexcept;

```

In [stopsource.nonmembers], edit as follows:

```

[[nodiscard]] friend bool
operator==(const stop_source& lhs, const stop_source& rhs) noexcept;

```

In [thread.jthread.class.general], edit as follows:

```

[[nodiscard]] bool joinable() const noexcept;
void join();
void detach();
[[nodiscard]] id get_id() const noexcept;
[[nodiscard]] native_handle_type native_handle();
.
.
[[nodiscard]] stop_source get_stop_source() noexcept;
[[nodiscard]] stop_token get_stop_token() const noexcept;
.
.
[[nodiscard]] static unsigned int hardware_concurrency() noexcept;

```

In [thread.jthread.mem], edit as follows:

```

[[nodiscard]] bool joinable() const noexcept;

```

In [thread.jthread.stop], edit as follows:

```

[[nodiscard]] stop_source get_stop_source() noexcept;
.
.
[[nodiscard]] stop_token get_stop_token() const noexcept;

```

In [thread.jthread.static], edit as follows:

```

[[nodiscard]] static unsigned int hardware_concurrency() noexcept;

```

In [thread.barrier.class], edit as follows:

```

[[nodiscard]] arrival_token arrive(ptrdiff_t update = 1);
.
.
[[nodiscard]] arrival_token arrive(ptrdiff_t update = 1);

```

In [future.syn], edit as follows:

```

template<class F, class... Args>
[[nodiscard]] future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
async(F&& f, Args&&... args);
template<class F, class... Args>
[[nodiscard]] future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
async(launch policy, F&& f, Args&&... args);

```

In [futures.async], edit as follows:

```

template<class F, class... Args>
[[nodiscard]] future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
async(F&& f, Args&&... args);
.
.
template<class F, class... Args>
[[nodiscard]] future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
async(launch policy, F&& f, Args&&... args);

```

In [saferecl.hp.holder.general], edit as follows:

```
[[nodiscard]] bool empty() const noexcept;
```

and in [saferecl.hp.holder.mem], same edit for the member declaration's specification:

```
[[nodiscard]] bool empty() const noexcept;
```